

NAME: M.Greeshma

COURSE: Design and Analysis of Algorithms

CODE: CSA0619

REPORT

Q23. Case Study: Contact Search in Mobile Application (Binary Search) A mobile application stores contacts in sorted order and must retrieve them quickly during user input. Analyze how Binary Search improves search efficiency in this scenario. Identify requirements such as sorted data and quick response time. Apply divide and conquer concepts

to explain the search process. Analyze time complexity and design a solution, justifying its optimality.

Sol..

Code :

```
def binary_search(contacts, target):  
    """  
    Perform binary search on a sorted list of contacts.  
    :param contacts: List of contact names (sorted  
alphabetically).  
    :param target: Contact name to search for.  
    :return: Index of target if found, else -1.  
    """  
  
    left, right = 0, len(contacts) - 1  
  
    while left <= right:  
        mid = (left + right) // 2  
        if contacts[mid] == target:  
            return mid # Contact found  
        elif contacts[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1 # Contact not found
```

Example usage

```
contacts = ["Aarav", "Ananya", "Kiran", "Meera", "Ravi",  
"Sita", "Vikram"]
```

```
target = "Ravi"
```

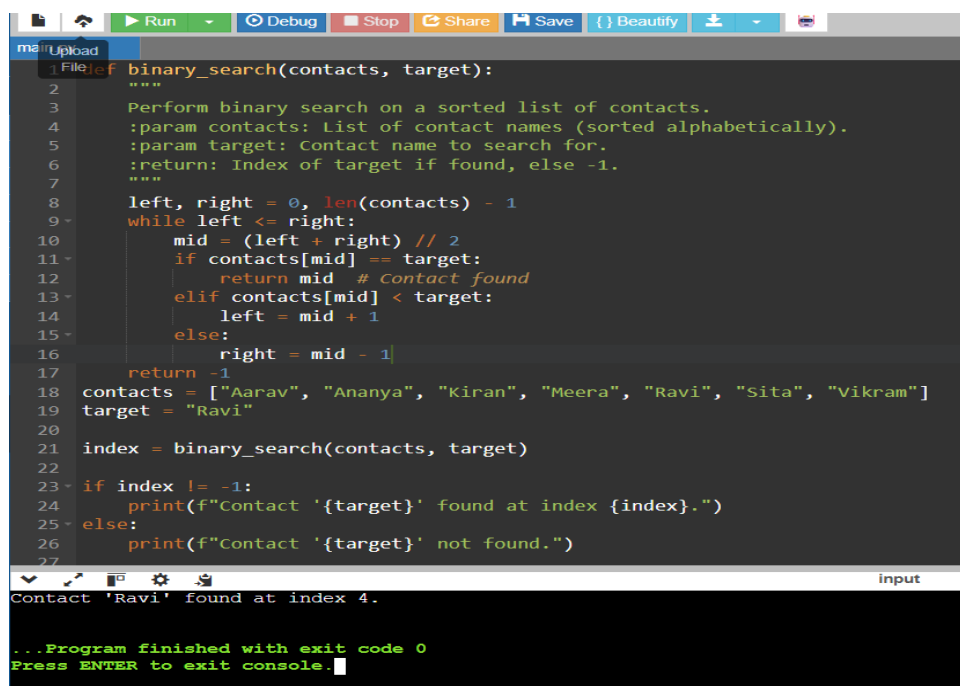
```
index = binary_search(contacts, target)
```

```
if index != -1:
```

```
    print(f"Contact '{target}' found at index {index}.")
```

```
else:
```

```
    print(f"Contact '{target}' not found.")
```



```
def binary_search(contacts, target):  
    """  
    Perform binary search on a sorted list of contacts.  
    :param contacts: List of contact names (sorted alphabetically).  
    :param target: Contact name to search for.  
    :return: Index of target if found, else -1.  
    """  
    left, right = 0, len(contacts) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if contacts[mid] == target:  
            return mid # Contact found  
        elif contacts[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1  
contacts = ["Aarav", "Ananya", "Kiran", "Meera", "Ravi", "Sita", "Vikram"]  
target = "Ravi"  
index = binary_search(contacts, target)  
if index != -1:  
    print(f"Contact '{target}' found at index {index}.")  
else:  
    print(f"Contact '{target}' not found.")
```

Contact 'Ravi' found at index 4.

...Program finished with exit code 0
Press ENTER to exit console.

Why Binary Search?

- **Efficiency:** Linear search would require checking each contact one by one, which is slow for large contact lists.
- **Binary Search** leverages sorted data to cut the search space in half at each step, drastically reducing lookup time.
- **Quick response time** is critical in mobile apps to ensure smooth user experience.

Requirements

- **Sorted Data:** Contacts must be stored in ascending order (e.g., alphabetically).
- **Fast Response:** Search results should appear instantly as the user types.
- **Scalability:** Must handle thousands of contacts without lag.
- **Divide & Conquer Principle:** The algorithm repeatedly divides the dataset into halves until the target is found.

Search Process (Divide & Conquer)

1. Start with the middle contact in the sorted list.
2. Compare the search query with the middle contact:
 - If equal → contact found.
 - If query < middle → search left half.
 - If query > middle → search right half.
3. Repeat until the contact is found or the search space is empty.

Example: Searching for "Ravi" in a list of 1000 contacts:

- Step 1: Check contact at position 500.
- Step 2: Narrow down to 250, then 125, then 62...
- Within ~10 steps, the contact is found (instead of checking all 1000).

Time Complexity

- **Binary Search:**
- **Linear Search:**

For contacts:

- Linear Search → up to 1,000,000 comparisons.
- Binary Search → about 20 comparisons.

This exponential improvement ensures **real-time responsiveness**.

Solution Design

- Store contacts in a **sorted array or balanced tree**.
- Use **binary search** for exact matches.

- For **prefix search** (e.g., typing "Ra" to find "Ravi"), apply binary search to find the first occurrence, then expand sequentially to fetch all matches.
- Optimize with **tries or hash maps** for predictive text, but binary search remains optimal for sorted lists.

Key Result Points

1. Sorted Data Requirement

- Binary Search only works on sorted contact lists (alphabetical order).
- Ensures predictable and efficient searching.

2. Divide & Conquer Approach

- The search space is halved at each step.
- Reduces comparisons drastically compared to linear search.

3. Time Complexity

- Binary Search:
- Linear Search:
- For 1,000,000 contacts → ~20 steps vs. 1,000,000 steps.

4. Quick Response Time

- Essential for mobile applications where users expect instant results.
- Binary Search guarantees responsiveness even with large datasets.

5. Optimality

- Minimal comparisons make it the most efficient method for sorted lists.
- Balances speed and simplicity, easy to implement in mobile apps.

6. Scalability

- Handles thousands or millions of contacts without performance issues.
- Future-proof as user data grows.

7. Extension for Prefix Search

- Binary Search can locate the first occurrence of a

prefix (e.g., "Ra").

- Sequential expansion retrieves all matching contacts quickly.
- Supports predictive text and auto-suggestions.